

**DEPARTMENT OF COMPUTER & INFORMATION SYSTEMS ENGINEERING
BACHELORS IN COMPUTER SYSTEMS ENGINEERING**

Course Code: CS-116

Course Title: Object Oriented Programming

Complex Engineering Problem

FE Batch 2025, Spring Semester 2026

Grading Rubric

TERM PROJECT

Student No.	Name	Roll No.
S1	RUMAISA WASEEM	CS-25052
S2	ALEENA KHALID	CS-25056
S3	LAIBA QASIM	CS-25069
S4		

CRITERIA AND SCALES					Marks Obtained			
					S1	S2	S3	S4
Criterion 1: Does the application meet the desired specifications and produce the desired outputs? (CPA-1, CPA-3)								
1	2	3	4					
The application does not meet the desired specifications and is producing incorrect outputs.	The application partially meets the desired specifications and is producing incorrect or partially correct outputs.	The application meets the desired specifications but is producing incorrect or partially correct outputs.	The application meets all the desired specifications and is producing correct outputs.					
Criterion 2: How well is the code organization?								
1	2	3	4					
The code is poorly organized and very difficult to read.	The code is readable only to someone who knows what it is supposed to be doing.	Some part of the code is well organized, while some part is difficult to follow.	The code is well organized and very easy to follow.					
Criterion 3: How friendly is the application interface? (CPA-1, CPA-3)								
1	2	3	4					
The application interface is difficult to understand and use.	The application interface is easy to understand and but not that comfortable to use.	The application interface is very easy to understand and use.	The application interface is very interesting/ innovative and easy to understand and use.					
Criterion 4: How does the student performed individually and as a team member? (CPA-2, CPA-3)								
1	2	3	4					
The student did not work on the assigned task.	The student worked on the assigned task, and accomplished goals partially.	The student worked on the assigned task, and accomplished goals satisfactorily.	The student worked on the assigned task, and accomplished goals beyond expectations.					
Criterion 5: Does the report adhere to the given format and requirements?								
1	2	3	4					
The report does not contain the required information and is formatted poorly.	The report contains the required information only partially but is formatted well.	The report contains all the required information but is formatted poorly.	The report contains all the required information and completely adheres to the given format.					
Total Marks:								

INDEX

Contents

1. PROBLEM DESCRIPTION	4
2. DISTINGUISHING FEATURES OF PROJECT AND LIST OF OBJECT-ORIENTED FEATURES IMPLEMENTED.....	5
2.1 Encapsulation	5
2.2 Abstraction.....	5
2.3 Inheritance.....	5
2.4 Polymorphism.....	6
2.5 Operator Overloading.....	6
2.6 File Handling	6
2.7 Composition and Aggregation.....	7
2.8 Exception Handling.....	7
2.9 Modular Class-Based Design.....	7
2.10 Secure Authentication and Validation	8
2.11 Report Generation and Analytics.....	8
2.12 Real-World System Simulation	8
2.13 Reservation, Waitlist, and Notification System	9
2.14 Password Hashing and Salting.....	9
2.15 Ratings and Reviews System	9
3. FLOW OF PROJECT AND CLASS DIAGRAM.....	10
3.1 Program Startup and Initialization	10
3.2 Super Admin Initialization	10
3.3 Main Portal Interface	10
3.4 User Registration Flow	11
3.5 User Login and Authentication	11
3.6 Member Dashboard Flow.....	12
3.7 Resource Browsing and Searching	12
3.8 Borrowing Process	13
3.9 Ebook Access Flow	13
3.10 Returning Process	13
3.11 Reservation and Waitlist Flow	14
3.12 Notification System	14
3.13 Fine Management System	14
3.14 Rating and Review System	15
3.15 Admin Dashboard Flow.....	15

3.16 Resource Management by Admin	15
3.17 Report Generation and Analytics	16
3.18 File Handling and Data Persistence.....	16
3.19 Program Termination	16
3.20 CLASS DIAGRAM.....	18
4. MOST CHALLENGING PART WHILE WORKING ON THE PROJECT	19
4.1 Integration of Multiple Object-Oriented Modules.....	19
4.2 Secure Authentication Using Password Hashing and Salting	19
4.3 Comprehensive Input Validation System.....	19
4.4 Role-Based Access Control and Admin Privilege Management	19
4.5 Separation of Member and Admin Functionalities	20
4.6 Inheritance, Polymorphism, and File Handling	20
4.7 Borrowing and Returning System Complexity	20
4.8 Maintaining Data Consistency Across Multiple Files	20
4.9 Debugging and Runtime Error Handling	20
4.10 Overall Project Complexity.....	20
4.11 Module Integration and Coordination	21
4.12 Runtime Polymorphism and Dynamic Memory Management	21
4.13 Borrowing System Validation and Record Management	21
4.14 Debugging Object Relationships and File Serialization	21
4.15 Final Project Integration and Stability	21
4. NEW CONCEPTS LEARNT IN C++ WHILE WORKING ON THE PROJECT	22
6. TEST CASES.....	31
Test Case 1 – System Initialization	31
Test Case 2 - Member Registration	32
Test Case 3 - Admin Registration (restricted)	33
Test Case 4 Member Login and Privileges.....	38
Test Case 5 Report Generation System	42
7. INDIVIDUAL CONTRIBUTIONS OF EACH MEMBER IN THE PROJECT	44
User & Account Management Module	44
Resource Management Module.....	44
Borrowing, Fine Management & System Integration Module	45

1. PROBLEM DESCRIPTION

The purpose of this project is to design and implement a complete **Library Management System** using **Object-Oriented Programming (OOP) in C++**. The system is developed in a modular and structured manner using multiple interconnected classes to simulate a real-world library environment with proper user roles, resource handling, and transaction management.

In traditional library systems, operations such as maintaining book records, managing user accounts, tracking borrowed resources, and calculating fines are often handled manually. This manual approach becomes inefficient, time-consuming, and error-prone as the number of users and resources increases. Problems such as data inconsistency, duplicate records, misplaced information, and difficulty in tracking overdue books make manual systems unreliable and difficult to maintain.

To overcome these limitations, this project provides a fully automated and object-oriented solution. The system is divided into multiple modules, including User Management, Resource Management, Borrowing Management, Fine Management, Notifications, and Report Generation, where each module is implemented through separate classes and integrated into a centralized library system.

The system supports two types of users, mainly Members and Admins, each with specific roles and permissions. Admins are responsible for managing resources, handling user accounts, generating reports, monitoring activities, and performing system-level operations. Members can search resources, borrow and return books, make reservations, view notifications, pay fines, and access their profiles.

The system is designed to handle the following core functionalities:

- Secure user registration and login system with proper validation and authentication
- Role-based access control for Members and Admins
- Efficient management of library resources including Books and E-books using inheritance
- Automated borrowing and returning system with due date tracking
- Reservation and waitlist management for unavailable resources
- Fine calculation and management system for overdue resources
- Notification system for borrowing activities, reservations, and fines
- Persistent data storage using file handling in C++ to maintain records permanently
- Report and analytics generation for users, resources, and borrowing history
- Modular system design using multiple interconnected classes integrated through a central LibraryManager system

The implementation follows strong Object-Oriented Programming principles including encapsulation, inheritance, abstraction, polymorphism, composition, operator overloading, and exception handling. These concepts make the system more structured, reusable, secure, and easier to maintain. The project also utilizes advanced C++ features such as STL containers, file handling, dynamic memory management, runtime polymorphism, and custom exception classes to improve system performance and reliability. The main objective of this project is to provide an efficient, scalable, secure, and user-friendly library management solution that reduces manual workload while improving accuracy, reliability, and performance of library operations.

Overall, this project demonstrates a complete real-world implementation of Object-Oriented Programming concepts by organizing the program into modular components and integrating them into a fully functional and practical Library Management System.

2. DISTINGUISHING FEATURES OF PROJECT AND LIST OF OBJECT-ORIENTED FEATURES IMPLEMENTED

This project is a fully structured and modular Object-Oriented Library Management System developed in C++. The distinguishing features and object-oriented concepts implemented in this project are discussed below:

2.1 Encapsulation

Encapsulation is implemented throughout the system by declaring class data members as private or protected and restricting direct access from outside the class. Controlled access is provided through public getter and setter functions. This ensures data security, controlled modification of records, and better maintainability of the system.

Sensitive information such as passwords, user credentials, account balances, borrowing history, and fine records are protected from unauthorized access.

Classes demonstrating encapsulation include:

- User
- Member
- Admin
- Resource
- BorrowRecord
- FineManager

2.2 Abstraction

Abstraction is achieved through abstract base classes and generalized interfaces that hide implementation details while exposing only essential functionalities. The User and Resource classes act as generalized parent classes containing virtual or pure virtual functions that define common behavior for derived classes.

This approach simplifies system usage and allows developers to focus on high-level operations instead of internal implementation details.

Examples include:

- User as a base class for Member and Admin
- Resource as a base class for Book and Ebook
- Generalized management through LibraryManager and UserManager.

2.3 Inheritance

Inheritance is extensively used to improve code reusability and reduce duplication. Common properties and functions are inherited from parent classes, while derived classes implement specialized behavior according to their requirements.

The inheritance hierarchy used in the project includes:

User Hierarchy

- User → Member
- User → Admin

Resource Hierarchy

- Resource → Book
- Resource → Ebook

This hierarchical structure makes the code more organized, extensible, and easier to maintain.

2.4 Polymorphism

Polymorphism is implemented using virtual functions and method overriding. Derived classes provide their own implementation of functions such as `displayProfile()` and `displayDetails()`, allowing runtime decision-making based on the actual object type.

This enables flexible handling of different object types through base class pointers and references.

Examples include:

- `displayDetails()` in Book and Ebook
- `displayProfile()` in Member and Admin
- Runtime handling of Resource objects using polymorphism

2.5 Operator Overloading

Operator overloading is implemented to provide meaningful comparison between objects. The equality operator (`=`) is overloaded in classes such as User and Resource to compare objects based on unique identifiers like `userID` and `resourceID`.

This improves code readability and simplifies object comparison operations within the system.

2.6 File Handling

The project uses file handling to permanently store and retrieve system data. Information related to users, resources, borrow records, reservations, fines, and notifications is stored in text files using `fstream`.

This ensures data persistence so that all records remain saved even after the program terminates.

File handling is used for:

- User records
- Resource records
- Borrowing history
- Reservation records
- Notifications
- Fine details

2.7 Composition and Aggregation

The project demonstrates strong object relationships through composition and aggregation.

The LibraryManager class acts as the central controller of the system and manages collections of:

- Resource objects
- BorrowRecord objects
- Notification objects
- User objects

Similarly, the UserManager class manages:

- User objects
- Member objects
- Admin objects

It also interacts with:

- SecurityUtils for authentication and password management
- Validator for input validation and verification

These relationships reflect real-world interactions between entities in a library environment and improve modular integration, security, maintainability, and efficient communication among different system components.

2.8 Exception Handling

Exception handling is implemented to manage runtime errors and invalid operations safely. Custom exception classes are used to prevent unexpected crashes and improve system reliability.

The system handles exceptions for:

- Invalid login attempts
- Incorrect password entries
- Duplicate registrations
- Exceeding borrowing limits
- Returning unavailable books
- Insufficient balance for fines
- Invalid admin registration keys
- File handling errors

This makes the system more secure, stable, and user-friendly.

2.9 Modular Class-Based Design

The entire project is divided into multiple independent but interconnected modules. Each module is responsible for a specific functionality, following the principle of separation of concerns.

The major modules include:

- User Management Module
- Resource Management Module
- Borrowing and Fine Management Module
- Reservation and Notification Module
- Report Generation Module
- Security and Validation Module
- Library Integration Module

This modular structure improves:

- Code readability
- Maintainability
- Scalability
- Debugging efficiency
- Future expansion capabilities

2.10 Secure Authentication and Validation

The system includes a secure authentication mechanism with proper validation and protected user credentials. Validation utilities are implemented to ensure correct user input and prevent invalid data entry.

Features include:

- Password validation
- Admin access key verification
- Duplicate account prevention
- Secure credential handling

2.11 Report Generation and Analytics

The project includes report generation and analytics features that allow administrators to monitor library activities effectively.

The system can generate:

- User reports
- Resource reports
- Borrowing history reports
- Overdue reports
- Fine reports
- System analytics summaries

These reports help in efficient library monitoring and management.

2.12 Real-World System Simulation

The project simulates a practical real-world library system where members can register, log in, search resources, borrow books, reserve unavailable items, return resources, and pay fines. Admins can manage resources, monitor users, generate reports, and control overall system operations. The

integration of multiple OOP concepts with real-world functionalities makes the project practical, scalable, and application-oriented.

2.13 Reservation, Waitlist, and Notification System

The project supports integrated reservation and waitlist management. Users can reserve unavailable resources and join waitlists. When a resource becomes available, the system automatically processes the next eligible user. Notifications are generated and stored for important events such as borrow/return updates, reservation alerts, fine notifications, and approval statuses.

2.14 Password Hashing and Salting

The project uses password hashing and salting to securely store user credentials instead of saving passwords in plain text. Unique salt values are added before hashing to improve security and protect against brute-force and dictionary attacks. This functionality is implemented through the SecurityUtils module to ensure secure authentication and credential management.

2.15 Ratings and Reviews System

Members can provide ratings and reviews for books using a 1–5 star system. The project enforces rating validation rules and review constraints, including limiting one review per user for each resource. The structure also supports admin moderation capabilities.

Overall, this project strongly demonstrates the practical implementation of Object-Oriented Programming concepts in a real-world scenario. By integrating advanced OOP principles, modular design, secure authentication, exception handling, and file management, the system provides a scalable, secure, efficient, and user-friendly Library Management Solution.

3. FLOW OF PROJECT AND CLASS DIAGRAM

3.1 Program Startup and Initialization

When the program starts, execution begins from the `main()` function inside `main.cpp`. The system enters a top-level try block and initializes the core objects required for the application.

The following objects are created during startup:

- UserManager
- ReportGenerator
- LibraryManager

The `UserManager` constructor automatically loads all user records from `users.txt`, including Members and Admins. Similarly, the `LibraryManager` loads:

- Resources
- Borrow records
- Notifications
- Reservations
- Waitlists

After loading data, the system rebuilds borrowing states, restores resource availability, updates member borrowing counts, and synchronizes records before saving the updated state back into files.

3.2 Super Admin Initialization

Before normal system operations begin, the program checks whether a Super Admin already exists in the system.

If no Super Admin exists:

- The system enters initialization mode.
- The user is prompted to register a Super Admin account.
- Admin access code verification is required.
- Password hashing and salting are applied before storing credentials.
- The Super Admin account is saved permanently in `users.txt`.

Without a Super Admin, the system cannot continue execution.

3.3 Main Portal Interface

After successful initialization, the Central Library Portal is displayed.

The following options are available:

1. Register as Member
2. Register as Admin
3. Login to Account
4. Exit Program

All inputs are validated using InputHelper and Validator classes to prevent invalid entries and runtime errors.

3.4 User Registration Flow

Member Registration

When a user selects Member Registration:

- A unique user ID is generated.
- Username uniqueness is verified.
- User information is collected and validated.
- Password is hashed securely using SecurityUtils.
- A Member object is created.
- Data is stored in users.txt.

The following information is collected:

- Username
- Password
- Email
- CNIC
- Phone number
- Address
- Initial balance

Admin Registration

When Admin Registration is selected:

- Admin access code verification is required.
- User details are validated.
- Different admin roles can be assigned:
 - Standard Admin
 - Librarian
 - Notification Admin
 - Fine Manager
 - Super Admin
- Password hashing and salting are applied.
- Admin data is stored securely.

The system allows multiple admin roles with different permissions and access privileges.

3.5 User Login and Authentication

When a user selects Login:

- Username and password are entered.
- UserManager verifies credentials.
- Password hashes are compared securely.
- Invalid login attempts are handled through exception handling.

After successful login:

- The system identifies the user role.
- Appropriate dashboard is displayed.

3.6 Member Dashboard Flow

If the logged-in user is a Member, the Member Dashboard appears.

Members can perform the following operations:

- View profile
- Browse available resources
- Search resources
- Borrow physical books
- Access ebooks
- Return resources
- Extend due dates
- View borrowing history
- Rate and review books
- Manage notifications
- Renew membership
- Add funds
- Reserve resources
- Logout

These functionalities interact with:

- LibraryManager
- BorrowRecord
- FineManager
- Notification system
- Resource hierarchy

3.7 Resource Browsing and Searching

Members can browse resources using:

- Categories
- Titles
- Authors
- ISBN
- Availability status

The system filters resources dynamically and displays:

- Available books
- Reserved books
- Digital resources

AdvancedSearch functionality is used to optimize searching and filtering operations.

3.8 Borrowing Process

When a member attempts to borrow a resource, the following checks are performed:

- Resource existence verification
- Availability check
- Borrowing limit validation
- Membership validity verification
- Outstanding fine verification
- Reservation conflict checking

If all validations succeed:

- BorrowRecord is created.
- Issue date and due date are assigned.
- Resource status changes to unavailable.
- Member borrow count increases.
- Notifications are generated.
- Data is saved into files.

The borrowing process is managed through:

- LibraryManager::borrowResource()
- Member
- Resource
- BorrowRecord

3.9 Ebook Access Flow

When accessing digital resources:

- System verifies digital availability.
- License availability is checked.
- Borrow record is generated.
- Download/access link is provided.

Ebooks are handled separately from physical books through runtime polymorphism using the Ebook derived class.

3.10 Returning Process

When a resource is returned:

- Borrow record is located.
- Return date is recorded.
- Overdue days are calculated.
- FineManager calculates overdue fines.
- Member balance is updated.
- Resource becomes available again.
- Waitlisted users are notified automatically.

The return process updates:

- Borrow records
- Member borrow count
- Notifications
- Resource availability

All changes are saved back into files.

3.11 Reservation and Waitlist Flow

If a resource is unavailable:

- Members can reserve the resource.
- Members can join the waitlist.
- Reservation data is stored in reservations.txt.
- Waitlist data is stored in waitlist.txt.

When a resource becomes available:

- First user in waitlist receives notification.
- Reservation priority is granted automatically.

This system improves fairness and resource management efficiency.

3.12 Notification System

The notification system automatically informs users regarding:

- Borrow approvals
- Due dates
- Fine alerts
- Book availability
- Reservation confirmations
- Return confirmations

Notifications are stored permanently and linked to Member accounts.

3.13 Fine Management System

The FineManager module handles overdue fine calculations.

The system:

- Calculates overdue charges
- Updates user balances
- Tracks pending fines
- Validates payments

Fine calculations are automatically triggered during the return process.

3.14 Rating and Review System

Members can:

- Rate books from 1–5
- Add reviews and feedback

The system validates ratings and stores them through the BookRating module.

This improves user interaction and resource evaluation.

3.15 Admin Dashboard Flow

If the logged-in user is an Admin, the Admin Dashboard is displayed according to assigned permissions.

Admins can perform:

- View all members
- Add resources
- Update resources
- Remove resources
- Process borrow requests
- Process returns
- Manage fines
- Generate reports
- View analytics
- Manage notifications
- View audit logs
- Manage admins
- Manage members

Different admin roles have different access privileges based on role-based permission handling.

3.16 Resource Management by Admin

Admins can:

- Add books and ebooks
- Update resource information
- Remove resources
- Manage availability
- Track inventory

Resources are managed using:

- Resource base class
- Book derived class
- Ebook derived class

Inheritance and polymorphism are heavily used in this module.

3.17 Report Generation and Analytics

The system generates:

- Member reports
- Borrow history reports
- Overdue reports
- Fine reports
- Inventory reports
- System statistics

Analytics help administrators monitor:

- Most borrowed books
- Active members
- Overdue resources
- Library activity statistics

This functionality is handled through:

- ReportGenerator
- AdminAnalytics

3.18 File Handling and Data Persistence

The entire system uses file handling for permanent data storage.

The following files are maintained:

- users.txt
- resources.txt
- borrow_records.txt
- notifications.txt
- reservations.txt
- waitlist.txt

All updates are automatically saved before program termination, ensuring persistence and consistency of records.

3.19 Program Termination

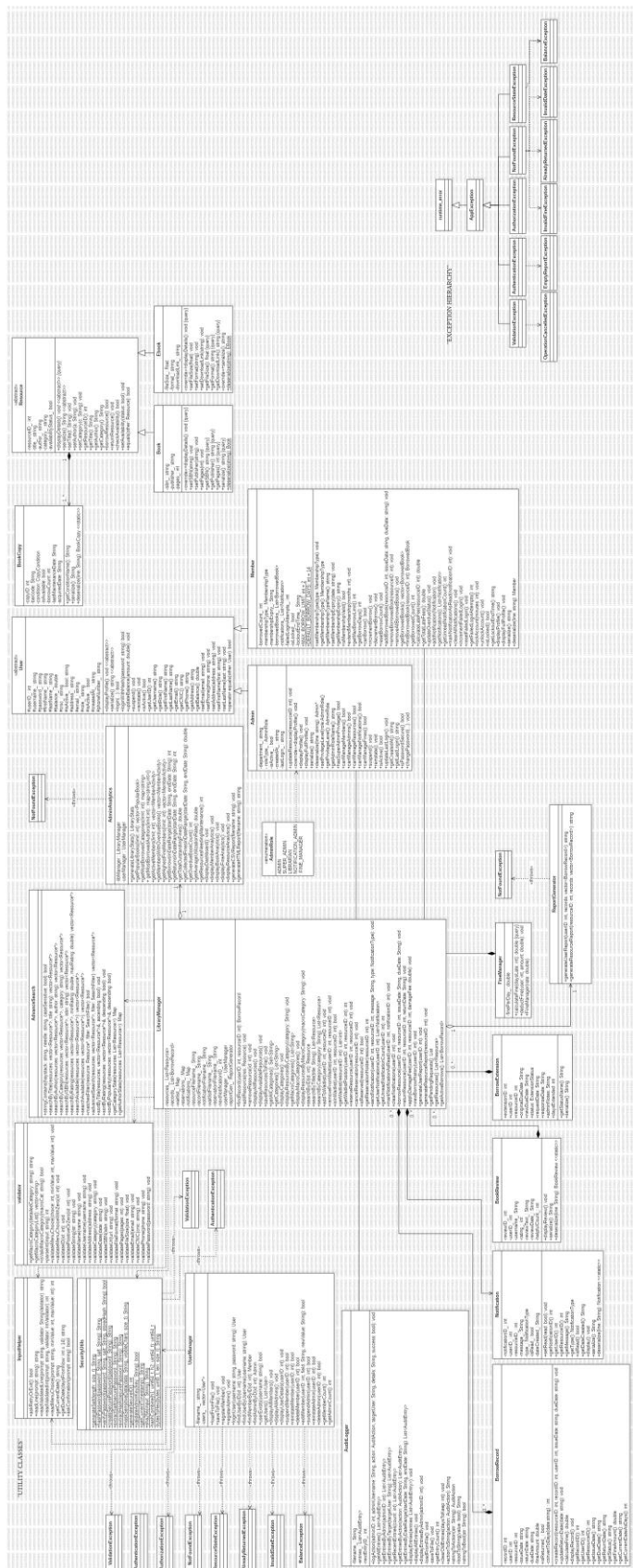
When the user selects Exit:

- Updated data is saved.
- Objects are destroyed safely.
- Memory is released.
- Program terminates successfully.

The system ensures that no data is lost before shutdown.

Overall, the program follows a complete modular workflow where all components interact efficiently through Object-Oriented Programming principles. The integration of authentication, role-based access control, borrowing management, notifications, file handling, analytics, and security mechanisms makes the Library Management System a realistic, scalable, and fully functional software solution.

3.20 CLASS DIAGRAM



4. MOST CHALLENGING PART WHILE WORKING ON THE PROJECT

4.1 Integration of Multiple Object-Oriented Modules

The most challenging part of this project was designing and integrating multiple object-oriented modules into a single fully functional system while maintaining proper communication and coordination between classes. Since the project was divided into several independent modules such as User Management, Resource Management, Borrowing & Fine Management, Security Management, and Report Generation, ensuring smooth interaction among all components required careful planning and implementation.

4.2 Secure Authentication Using Password Hashing and Salting

One of the most difficult and time-consuming tasks was implementing secure password hashing and salting mechanisms for user authentication. Instead of storing passwords in plain text, the system was designed to securely hash passwords along with salt values to improve security and prevent unauthorized access. Understanding how hashing algorithms, salting techniques, and secure credential storage work in C++ required significant research and testing. Ensuring that login verification correctly compared hashed passwords while maintaining data consistency was also a major challenge.

4.3 Comprehensive Input Validation System

Another challenging aspect was implementing comprehensive validation checks for every user input throughout the system. Since the project handles sensitive operations such as registration, login, borrowing, fine payment, and admin operations, strong validation was necessary to prevent invalid data entry and system misuse. Validation had to be applied for:

- Usernames and passwords
- Email addresses and contact details
- Resource IDs and borrowing limits
- Duplicate records
- Invalid menu choices
- Fine payment amounts
- Admin registration keys

Designing a validation system that handled all possible invalid inputs without crashing the program required careful exception handling and extensive testing.

4.4 Role-Based Access Control and Admin Privilege Management

Creating and managing multiple admin roles with different privileges was also one of the most difficult parts of the project. The system needed to implement role-based access control where different admins had different levels of authority and permissions. For example, some admins could manage resources while others could generate reports, manage users, or access analytics. Ensuring that each admin role only accessed authorized functionalities required careful privilege management and secure permission verification throughout the system.

4.5 Separation of Member and Admin Functionalities

The system also needed to clearly separate Member functionalities from Admin functionalities while ensuring that only authorized admins could perform sensitive operations such as:

- Adding or removing resources
- Managing users
- Generating reports
- Accessing analytics
- Controlling system-level operations
- Viewing borrowing histories
- Managing fines and notifications

Implementing secure admin registration keys, restricted access controls, multiple admin privileges, and role verification added additional complexity to the project.

4.6 Inheritance, Polymorphism, and File Handling

Another major challenge was handling inheritance and polymorphism together with file handling. Ensuring that objects of derived classes such as Member, Admin, Book, and Ebook were correctly stored, retrieved, and reconstructed from files required proper use of runtime polymorphism, dynamic memory allocation, and object casting techniques.

4.7 Borrowing and Returning System Complexity

The borrowing and returning system was also complex because multiple conditions needed to be verified simultaneously, including:

- Checking resource availability
- Verifying borrowing limits
- Managing reservations and waitlists
- Updating borrow records correctly
- Calculating overdue fines accurately
- Updating resource status dynamically

4.8 Maintaining Data Consistency Across Multiple Files

Managing consistency across multiple files such as users, resources, borrow records, fines, and notifications was another difficult task because even a small mismatch in stored data could lead to incorrect system behavior.

4.9 Debugging and Runtime Error Handling

Debugging issues related to pointers, object relationships, dynamic memory management, file serialization/deserialization, and runtime exceptions required extensive testing and debugging sessions.

4.10 Overall Project Complexity

Overall, the most challenging part of the project was implementing secure authentication using password hashing and salting, performing complete validation for every user input, creating multiple admin roles with secure privilege handling, and integrating all object-oriented modules into a single

cohesive and reliable system while maintaining proper OOP design principles and preventing runtime errors.

4.11 Module Integration and Coordination

The most challenging part of this project was designing and implementing the integration of multiple object-oriented modules into a single working system while maintaining proper communication between classes.

Since the project was divided into different parts such as User Management, Resource Management, and Borrowing & Fine Management, each module was developed independently. However, combining these modules into a fully functional system required careful planning and coordination to ensure that all classes worked together correctly without errors.

4.12 Runtime Polymorphism and Dynamic Memory Management

One of the major difficulties was handling inheritance and polymorphism together with file handling. Ensuring that objects of derived classes such as Member, Admin, Book, and Ebook were correctly stored, retrieved, and cast back to their original types required proper use of dynamic memory and runtime type identification.

4.13 Borrowing System Validation and Record Management

Another challenging aspect was implementing the borrowing and returning system, where multiple conditions had to be checked simultaneously, such as:

- Whether the resource is available
- Whether the member has reached the borrowing limit
- Proper creation and updating of borrow records
- Accurate calculation of fines for overdue items

Additionally, managing data consistency across different files (users, resources, and borrow records) was difficult because any mismatch could lead to incorrect system behavior.

4.14 Debugging Object Relationships and File Serialization

Debugging errors related to pointer management, object relationships, and file serialization/deserialization was also time-consuming and required careful testing.

4.15 Final Project Integration and Stability

Overall, the most challenging part was ensuring smooth integration of all modules into a single cohesive system while maintaining proper object-oriented design principles and avoiding runtime errors.

4. NEW CONCEPTS LEARNT IN C++ WHILE WORKING ON THE PROJECT

During the development of this project, several advanced and practical concepts of C++ were learned and applied beyond the basic programming level. These concepts helped in building a more structured, efficient, secure, and real-world application using Object-Oriented Programming principles and modular software design.

5.1 File Handling (Data Persistence)

One of the most important concepts learned was file handling in C++. It was used to store and retrieve data such as users, resources, borrow records, notifications, and fines. This ensured that all information remained saved even after the program was closed, making the system persistent and realistic.

Concepts learned include:

- Reading and writing files using `fstream`
- File serialization and deserialization
- Managing multiple data files
- Updating records dynamically
- Maintaining data consistency

5.2 Templates (Generic Programming)

Templates were learned as a way to create generic and reusable functions and classes. They allow writing code that can work with different data types without rewriting logic multiple times.

This concept helped in:

- Reducing code duplication
- Improving flexibility
- Creating reusable utilities
- Making the system more scalable

Templates improved understanding of generic programming in C++.

5.3 Mixins / Reusable Class Behavior (Conceptual Learning)

The concept of mixins was explored as a method to provide reusable functionality to multiple classes without relying heavily on deep inheritance hierarchies. Although used conceptually, it helped in understanding how modular and reusable behavior can be shared across unrelated classes.

This improved understanding of:

- Flexible software design
- Code modularity
- Reusable behaviors
- Alternative approaches to inheritance

5.4 Object-Oriented Design in Real Systems

The project provided a deeper understanding of how Object-Oriented Programming concepts are implemented in real-world software systems. Practical implementation of:

- Encapsulation
- Inheritance
- Abstraction
- Polymorphism
- Composition
- Aggregation

helped in designing a modular system where every class has a clear and specific responsibility.

This significantly improved software design and problem-solving skills.

5.5 Operator Overloading in Practical Use

The project helped in understanding how operator overloading can simplify object comparison and improve code readability.

For example:

- The equality operator (`=`) was overloaded to compare User and Resource objects based on their unique IDs.

This reduced complex comparison logic and made the code more intuitive and maintainable.

5.6 Dynamic Memory and Pointers

Working with multiple interconnected classes introduced practical use of:

- Pointers
- Dynamic memory allocation
- Base class pointers
- Runtime polymorphism

The project improved understanding of:

- Heap memory management
- Object creation and destruction
- Pointer relationships
- Dynamic object handling
- Memory optimization

This was especially important while handling collections of users and resources through base class pointers.

5.7 Enum Classes (Strongly Typed Enumerations)

Another important concept learned was the use of enum classes in C++. Enum classes provide strongly typed enumerations that improve code safety, readability, and organization compared to traditional enums.

Enum classes were useful for representing:

- User roles
- Resource types
- Borrowing status
- Notification types
- Admin privilege levels

Learning enum classes improved understanding of:

- Type safety
- Better code organization
- Scoped enumerations
- Cleaner switch-case handling

5.8 Multiple Header Files and C++ Libraries

During the project, many new standard header files and libraries of C++ were explored and used practically. This improved understanding of how different libraries support real-world software development.

Some important header files learned include:

- `<fstream>`
- `<sstream>`
- `<iomanip>`
- `<vector>`
- `<map>`
- `<set>`
- `<algorithm>`
- `<ctime>`

These libraries helped in:

- File management
- Data formatting
- Exception handling
- Container management
- Searching and sorting
- String manipulation
- Time management

This greatly expanded practical knowledge of the C++ Standard Library.

5.9 System Integration Skills

Another major learning experience was integrating multiple independent modules into a single working system while maintaining proper communication between classes.

Modules integrated include:

- User Management
- Resource Management
- Borrowing System
- Fine Management
- Notifications
- Report Generation
- Security and Validation

This improved understanding of:

- Modular software architecture
- Inter-class communication
- System scalability
- Dependency management
- Real-world software structure

5.10 Separation of Header (.h) and Implementation (.cpp) Files

A very important software engineering concept learned during the project was separating programs into:

- Header files (.h) containing class declarations, attributes, and function prototypes
- Implementation files (.cpp) containing actual function definitions and logic

This separation improved:

- Code readability
- Maintainability
- Reusability
- Project organization
- Debugging efficiency
- Team collaboration

It also provided practical exposure to professional software development practices used in large-scale real-world applications.

5.11 Secure Programming Concepts

The project also introduced practical secure programming techniques including:

- Password hashing
- Salting
- Input validation
- Secure authentication
- Role-based access control

This improved understanding of how security is implemented in real software systems to protect sensitive information and prevent unauthorized access.

Overall, this project significantly improved understanding of advanced C++ concepts including file handling, templates, enum classes, pointers, secure programming, modular architecture, reusable design patterns, and real-world object-oriented system development. It provided practical experience in building a complete software application rather than isolated academic programs, greatly improving both programming and software engineering skills.

During the development of this project, several advanced and practical concepts of C++ were learned and applied beyond the basic programming level. These concepts helped in building a more structured, efficient, and real-world application using Object-Oriented Programming.

5. FUTURE EXPANSIONS

The current Library Management System is developed as a modular and fully functional console-based application using Object-Oriented Programming in C++. Although the system already implements advanced OOP concepts and real-world functionalities, there are many future enhancements that can further improve its scalability, usability, automation, security, and real-world applicability.

The project has strong potential to evolve into a complete enterprise-level library management solution through the following advanced future expansions.

7.1 Graphical User Interface (GUI)

Currently, the system operates through a console-based interface. In the future, it can be upgraded into a modern GUI-based application using frameworks such as:

- Qt
- SFML
- wxWidgets

A graphical interface would provide:

- Better user interaction
- Improved navigation
- Interactive dashboards
- Visual analytics
- Professional software appearance

This would make the system more user-friendly and suitable for real-world deployment.

7.2 Database Integration

At present, the project stores data using text files. In future versions, the system can be integrated with professional databases such as:

- MySQL
- SQLite
- PostgreSQL
- MongoDB

Database integration would provide:

- Faster data access
- Better scalability
- Improved data consistency
- Enhanced security
- Multi-user support
- Efficient handling of large datasets

This would allow the system to function efficiently in universities, schools, and large public libraries.

7.3 Web-Based and Cloud-Based System

The project can be transformed into a web-based or cloud-based library management platform where users can access the system remotely through the internet.

Future cloud integration may include:

- Online user accounts
- Remote borrowing management
- Cloud backups
- Real-time synchronization
- Cross-device accessibility

This would enable users and administrators to access the library system from anywhere at any time.

7.4 Advanced Security Enhancements

Future versions of the system can include enterprise-level security mechanisms such as:

- Advanced password encryption
- Multi-factor authentication (MFA)
- Biometric authentication
- Role-based access control
- Activity monitoring and audit logs
- Secure session handling
- Database encryption
- CAPTCHA verification

These features would improve system protection against unauthorized access and cyber threats.

7.5 Multiple Admin Roles and Permission Hierarchy

The system can be expanded with a more advanced role-based management structure where different admins have different levels of permissions.

Examples include:

- Super Admin
- Resource Manager
- Finance Admin
- Report Manager
- Inventory Controller

Each admin role can have customized access rights and operational permissions, improving organizational management and security.

7.6 Mobile Application Development

A complete Android and iOS mobile application can be developed for the system.

Mobile features may include:

- Online book reservations
- Borrowing history tracking
- Push notifications
- QR code scanning
- Mobile login authentication
- Fine payment through mobile wallets

This would provide better accessibility and convenience for users.

7.7 Real-Time Notification System

The system can be upgraded with automated real-time notifications through:

- Email services
- SMS alerts
- Push notifications
- In-app notifications

Notifications may include:

- Due date reminders
- Overdue fine alerts
- Reservation confirmations
- Admin announcements
- Password reset alerts

This would improve communication between users and the library system.

7.8 Digital E-Library and Ebook Reader

Future versions may include:

- Built-in ebook reader
- PDF viewing support
- Digital note-taking
- Bookmark management
- Online ebook downloads

This would allow the system to support fully digital libraries.

7.9 Automated Backup and Recovery System

An automatic backup and recovery mechanism can be added to protect important system data.

Features may include:

- Scheduled backups
- Cloud backup synchronization
- Data recovery tools
- Backup encryption

This would improve reliability and prevent accidental data loss.

Overall, these future enhancements can transform the current console-based Library Management System into a highly advanced, scalable, intelligent, and enterprise-level software solution. By integrating modern technologies such as Artificial Intelligence, cloud computing, mobile applications, databases, advanced security, and real-time analytics, the system can become a complete smart library management platform suitable for real-world large-scale environments.

The current Library Management System is designed as a functional and modular desktop-based application using Object-Oriented Programming in C++. However, there are several possible improvements and extensions that can be added in the future to make the system more advanced, scalable, and user-friendly.

6. TEST CASES

Test Case 1 – System Initialization

TC1.1 First Run – Super Admin Registration Needed

```
PS C:\Users\WASIF\Desktop\test> g++ -std=c++17 main.cpp User.cpp Member.cpp Admin.cpp Usermanager.cpp Resource.cpp Book.cpp Ebook.cpp BorrowRecord.cpp FineManager.cpp ReportGenerator.cpp LibraryManager.cpp Validator.cpp InputHelper.cpp Notification.cpp AuditLogger.cpp SecurityUtils.cpp AdminAnalytics.cpp AdvancedSearch.cpp BookCopies.cpp BookRating.cpp BorrowExtension.cpp -o app
PS C:\Users\WASIF\Desktop\test> ./app

=====
SYSTEM INITIALIZATION REQUIRED
LIBRARY MANAGEMENT SYSTEM
=====

Welcome! This library system requires a Super Admin to be
registered as the first step before anyone else can use it.

IMPORTANT: Only ONE Super Admin is allowed in the entire system.
After you register as Super Admin, you can create other admins
with different roles (Librarian, Fine Manager, etc.)

Would you like to register as the Super Admin now? (y/n):
```

TC1.2 Super Admin Registration and Access Code “LibraryAdmin2026!”

```
Would you like to register as the Super Admin now? (y/n): y
Username: RumaisaWaseem_
Password: Neduet@52
First Name: Rumaisa
Last Name: Waseem
Email: rumaisawaseem52@gmail.com
CNIC (XXXXX-XXXXXXX-X): 42602-0334741-6
Phone (03XXXXXXXXX): 03312445480
Address: National Highway, Karachi.

To register as an admin, you must enter the shared company admin access code.
This code is issued by the library system owner and is required for all admin registrations.
You have 3 attempts to enter the correct code.
Admin access code: LibraryAdmin2026
Invalid admin access code. 2 attempt(s) remaining. Please contact the library system owner for the correct code.
Admin access code: LibraryAdmin2026!

*** SYSTEM INITIALIZATION ***
As the first administrator, you are being registered as a SUPER ADMIN.
After this registration, only super admins can register other admins.
```

SUCCESSFUL REGISTRATION :

```
=====
ADMIN REGISTRATION SUCCESSFUL
=====
User ID: 1
Username: RumaisaWaseem_
Name: Rumaisa Waseem
Role: Super Admin

Your admin account has been created successfully.

System initialization complete!
You can now log in and manage the library system.

=====
LIBRARY MANAGEMENT SYSTEM
Central Library Portal
=====

Please select an option to continue:
1. Register as a Member
2. Register as an Admin (Super Admin use only)
3. Login to your account
0. Exit

=====
Enter your choice:
```

TC1.3 Duplicate Super Admin Registration Handling

```
=====
LIBRARY MANAGEMENT SYSTEM
Central Library Portal
=====

Please select an option to continue:
1. Register as a Member
2. Register as an Admin (Super Admin use only)
3. Login to your account
0. Exit

=====
Enter your choice: 2

=====
ADMIN REGISTRATION RESTRICTED
=====
Only SUPER ADMINS can register other admins.
If you need to register a new admin, please:
1. Log in as the Super Admin
2. Navigate to Member Management
3. Use the admin registration option

This restriction ensures only authorized personnel
can grant admin privileges in the system.
=====
```

Test Case 2 - Member Registration

TC2.1 Member Registration and Input Validation

```
=====
LIBRARY MANAGEMENT SYSTEM
Central Library Portal
=====

Please select an option to continue:
1. Register as a Member
2. Register as an Admin (Super Admin use only)
3. Login to your account
0. Exit

=====
Enter your choice: 1
Username: Mahnoor@
Error: Validation Error:
Username must be 6-30 characters, start with a letter, and contain only letters, numbers, underscores, or dots.
Would you like to retry the input or exit this operation? (e/r): R
Username: MahnoorNaem54_
Password: Neduet
Error: Validation Error:
Password must be 8-64 characters with at least one uppercase letter, one lowercase letter, one number, and one special character. Spaces are not allowed.
Would you like to retry the input or exit this operation? (e/r): r
Password: █
```

Invalid phone number check

```
Phone (03XXXXXXXXX): 0345665478
Error: Validation Error:
Phone number must be 11 digits starting with 03 (e.g., 03001234567).
Would you like to retry the input or exit this operation? (e/r): r
Phone (03XXXXXXXXX): 03345335480
```

TC2.2 Valid Member Registration

```
=====
LIBRARY MANAGEMENT SYSTEM
Central Library Portal
=====

Please select an option to continue:
1. Register as a Member
2. Register as an Admin (Super Admin use only)
3. Login to your account
0. Exit

=====
Enter your choice: 1
Username: ManahilImran61_
Password: Neduet@61
First Name: Manahil
Last Name: Imran
Email: manahilimran61@gmail.com
CNIC (XXXXX-XXXXXX-X): 42602-4567893-8
Phone (03XXXXXXXXX): 03456788987
Address: Shah-Faisal, Karachi.
Initial account balance for future use (Rs.) [Press Enter for 0]: 500
```



```
=====
MEMBER REGISTRATION SUCCESSFUL
=====
User ID: 3
Username: ManahilImran61_
Name: Manahil Imran

Your account has been created successfully.
You can now log in with your credentials.
```

TC2.3 Duplicate Username Handling

Already exists:

```
=====
MEMBER REGISTRATION SUCCESSFUL
=====
User ID: 2
Username: MahnoorNaeem54_
Name: Mahnoor Naeem

Your account has been created successfully.
You can now log in with your credentials.
```

Duplication error message

```
=====
LIBRARY MANAGEMENT SYSTEM
Central Library Portal
=====

Please select an option to continue:
1. Register as a Member
2. Register as an Admin (Super Admin use only)
3. Login to your account
0. Exit

=====
Enter your choice: 1
Username: MahnoorNaeem54_
Error: Validation Error:
This username is already taken. Please choose a different username.
```

Test Case 3 - Admin Registration (restricted)

TC3.1 Restricted Admin Registration Access

```
=====
LIBRARY MANAGEMENT SYSTEM
Central Library Portal
=====

Please select an option to continue:
1. Register as a Member
2. Register as an Admin (Super Admin use only)
3. Login to your account
0. Exit

=====
Enter your choice: 2

=====
ADMIN REGISTRATION RESTRICTED
=====
Only SUPER ADMINS can register other admins.
If you need to register a new admin, please:
1. Log in as the Super Admin
2. Navigate to Member Management
3. Use the admin registration option

This restriction ensures only authorized personnel
can grant admin privileges in the system.
```

TC3.2 Super Admin Login and Dashboard

```
=====
LIBRARY MANAGEMENT SYSTEM
Central Library Portal
=====

Please select an option to continue:
1. Register as a Member
2. Register as an Admin (Super Admin use only)
3. Login to your account
0. Exit

=====
Enter your choice: 3

Username: RumaisaWaseem_
Password: Neduet@52

Login successful! Welcome, Rumaisa Waseem!
```

Dashboard

```
=====
ADMIN DASHBOARD
=====
[SUPER ADMIN MODE]

=====
1. View My Profile
2. View All Members
3. View All Resources
4. Process Borrow Request
5. Process Return Request
6. View Pending Requests
7. View Member Borrow History
8. Generate All Member Borrow Reports
9. Generate Issued/Overdue Resource Report
10. Notification Management
11. Resource Management
12. Fine Management
13. Member Management
14. Analytics Dashboard
15. View Audit Log
16. System Statistics
17. Admin Management
0. Logout
Choice: |
```

TC3.3 Super Admin Privileges

View all members registered on the system

```
Choice: 2

===== MEMBER PROFILE =====
User ID       : 2
Role          : Member
Username      : MahnoorNaeem54_
Full Name     : Mahnoor Naeem
Email        : mahnoornaem54@gmail.com
CNIC          : 42602-3456789-9
Phone        : 03345335480
Address       : Maskan, Karachi.
Membership Type : Silver
Membership Expiry: 2027-05-14
Borrowed Items : 0 / 2
Account Credit : Rs. 2000
-----

===== MEMBER PROFILE =====
User ID       : 3
Role          : Member
Username      : ManahilImran61_
Full Name     : Manahil Imran
Email        : manahilimran61@gmail.com
CNIC          : 42602-4567893-8
Phone        : 03456788987
Address       : Shah-Faisal, Karachi.
Membership Type : Silver
Membership Expiry: 2027-05-14
Borrowed Items : 0 / 2
Account Credit : Rs. 500
-----
```

View all resources on the system

Choice: 3

```
=====
PRINTED BOOK DETAILS
=====
ID: 1
Title: Clean Code
Author: Robert Martin
Category: Software Engineering
Status: Available
ISBN: 9780132350884
Publisher: Prentice Hall
Pages: 464
-----

=====
PRINTED BOOK DETAILS
=====
ID: 2
Title: The Pragmatic Programmer
Author: Andrew Hunt
Category: Software Engineering
Status: Available
ISBN: 9780135957059
Publisher: Addison Wesley
Pages: 352
-----

=====
PRINTED BOOK DETAILS
=====
ID: 3
Title: Design Patterns
Author: Erich Gamma
```

Member management functions

13. Member Management

```
Choice: 13

=====
MEMBER MANAGEMENT
=====
1. View Member Details
2. Suspend Member
3. Reinstate Member
4. Delete Member
5. Edit Member
6. View Member History
0. Back
Choice: 4
Member User ID: 2
WARNING: This will PERMANENTLY delete the member. Are you sure? (y/n): y

Member deleted successfully.
Member deleted.

=====
MEMBER MANAGEMENT
=====
1. View Member Details
2. Suspend Member
3. Reinstate Member
4. Delete Member
5. Edit Member
6. View Member History
0. Back
Choice: 1
Member User ID: 2
User not found.
```

Resource management functions

11. Resource Management

Adding a resource

```
Choice: 11
=====
RESOURCE MANAGEMENT
=====
1. Add New Resource
2. Remove Resource
3. Update Resource
4. View All Resources
0. Back
Choice: 1
=====
ADD NEW RESOURCE
=====
1. Printed Book
2. EBook
0. Back
Choice: 1
Title: A thousand splendid suns
Author: Ali Khan
Category: Emotional
ISBN (10 or 13 digits): 1234567891234
Publisher: Pager.com
Pages: 500
Physical location / branch: 7

Resource added successfully!
Resource ID: 51

New Resource ID: 51
=====
```

Added successfully , verification

```
=====
RESOURCE MANAGEMENT
=====
1. Add New Resource
2. Remove Resource
3. Update Resource
4. View All Resources
0. Back
Choice: 1
=====
ADD NEW RESOURCE
=====
1. Printed Book
2. EBook
0. Back
Choice: 1
Title: A thousand splendid suns
Author: Ali Khan
Category: Emotional
ISBN (10 or 13 digits): 1234567891234
Publisher: Pager.com
Pages: 500
Physical location / branch: 7

Resource added successfully!
Resource ID: 51

New Resource ID: 51
```

```
=====
RESOURCE MANAGEMENT
=====
1. Add New Resource
2. Remove Resource
3. Update Resource
4. View All Resources
0. Back
Choice: 4
```

```
=====
PRINTED BOOK DETAILS
=====
ID: 51
Title: A thousand splendid suns
Author: Ali Khan
Category: Emotional
Status: Available
ISBN: 1234567891234
Publisher: Pager.com
Pages: 500
-----
```

Deleting a resource

```
=====
RESOURCE MANAGEMENT
=====
1. Add New Resource
2. Remove Resource
3. Update Resource
4. View All Resources
0. Back
Choice: 2
Resource ID to remove: 51
Are you sure you want to remove this resource? (y/n): y

Resource removed successfully!

Resource removed.
```

TC3.4 Multiple Admin Role Registration

```
17. Admin Management
0. Logout
Choice: 17
```

Registration of a Librarian – Type of an Admin

```
=====
ADMIN MANAGEMENT
=====
1. Register New Admin
2. View All Admins
3. Suspend Admin
4. Reinstate Admin
5. Delete Admin
0. Back
Choice: 1
Username: AleenaKhalid56_
Password: Neduet@56
First Name: Aleena
Last Name: Khalid
Email: aleenakhalid56@gmail.com
CNIC (XXXXX-XXXXXX-X): 42602-3456789-8
Phone (03XXXXXXXXX): 03246755438
Address: Maskan, Karachi.

To register as an admin, you must enter the shared company admin access code.
This code is issued by the library system owner and is required for all admin registrations.
You have 3 attempts to enter the correct code.
Admin access code: LibraryAdmin2026!

Select admin role:
1. Standard Admin
2. Librarian
3. Notification Admin
4. Fine Manager
5. Super Admin
Choice: 3
```

```
=====
ADMIN REGISTRATION SUCCESSFUL
=====
User ID: 4
Username: AleenaKhalid56_
Name: Aleena Khalid
Role: Notification Admin

Your admin account has been created successfully.
```

Test Case 4 Member Login and Privileges

```
=====
LIBRARY MANAGEMENT SYSTEM
Central Library Portal
=====

Please select an option to continue:
1. Register as a Member
2. Register as an Admin (Super Admin use only)
3. Login to your account
0. Exit

=====
Enter your choice: 3

Username: ManahilImran61_
Password: Neduet@61

Login successful! Welcome, Manahil Imran!

--- Membership Info ---
Type: Silver | Borrow Limit: 0/2 | Expiry: 2027-05-14
Account Credit: Rs. 500
```

Member Dashboard

```
=====
MEMBER DASHBOARD
=====

Welcome back, Manahil Imran!
Membership: Silver
Borrowed Books: 0/2
Account Balance: Rs. 500 (Credit)

=====
RESOURCE MANAGEMENT
1. View Profile
2. Browse Available Resources
3. Search Resources
4. Borrow Physical Resource
5. Access Digital Resource
6. Return Resource
7. Extend Due Date
8. My Borrow History
9. Rate & Review Books
10. Notifications

ACCOUNT MANAGEMENT
11. Renew Membership
12. Add Funds
13. Reserve Resource

0. Logout
=====
Enter your choice: █
```

More Categories can be added by Librarian

```
=====
BROWSE BY CATEGORY
=====
1. Computer Science & IT
2. Data Science & Artificial Intelligence
0. Back to Menu
Select Your Desired Option: 0
```

Browse resources by various means:

```
Enter your choice: 3

=====
ADVANCED SEARCH
=====
1. Search by Title
2. Search by Author
3. Search by Category
4. Search by ISBN
5. View Unavailable Books & Request to Borrow
0. Back
Search Option: 1
Title: database
```

By title:

```

== SEARCH RESULTS (2) ==

[1]
=====
      PRINTED BOOK DETAILS
=====
ID: 9
Title: Database System Concepts
Author: Abraham Silberschatz
Category: Databases
Status: Available
ISBN: 9788073523323
Publisher: McGraw Hill
Pages: 1376

[2]
=====
      EBOOK DETAILS
=====
ID: 44
Title: Fundamentals of Database Systems
Author: Ramez Elmasri
Category: Databases
Status: Available
File Size (MB): 7.2
Format: EPUB
Download Link: https://example.com/ebooks/fundamentals-of-database-systems
Available Licenses: 1 / 1
Access Type: Online

```

Accessing digital books

5. Access Digital Resource

[illegible]

Notification Received

```
10. Notifications (1 new)

ACCOUNT MANAGEMENT
11. Renew Membership
12. Add Funds
13. Reserve Resource

0. Logout
=====
Enter your choice: 10

=====
YOUR NOTIFICATIONS
=====

[1] [*] BORROW APPROVED - 2026-05-14
You have successfully borrowed 'Fundamentals of Database Systems'. Due date: 2026-05-28. Please return on time to avoid late fees.
```

Borrowing a Book Successfully

4. Borrow Physical Resource

Borrow Resource 1

```
=====
Enter your choice: 4
Physical Resource ID to borrow: 3

>>> BORROW CONFIRMATION <<<

=====
PRINTED BOOK DETAILS
=====
ID: 3
Title: Design Patterns
Author: Erich Gamma
Category: Software Engineering
Status: Available
ISBN: 9780201633610
Publisher: Addison Wesley
Pages: 395
Are you sure you want to borrow this book? (y/n): y

>>> BORROW DETAILS <<<
Issue Date: 2026-06-16
Due Date: 2026-06-30
Borrow successful!
  Book: Design Patterns
  Due Date: 2026-06-30
  Late Fee: Rs. 10 per day after due date

Borrow processed successfully!
A notification has been sent to your notifications tab.
=====
```

Notification Received

```
[5] [*] BORROW APPROVED - 2026-06-16
You have successfully borrowed 'Design Patterns'. Due date: 2026-06-30. Please return on time to avoid late fees.
-----
```

Total Resources Borrowed

```
=====
MEMBER DASHBOARD
=====
Welcome back, Manahil Imran!
Membership: Silver
Borrowed Books: 2/2
Account Balance: Rs. 340 (Credit)
```

Borrowing Limit Validation

```
Enter your choice: 4
Physical Resource ID to borrow: 2

>>> BORROW CONFIRMATION <<<

=====
PRINTED BOOK DETAILS
=====
ID: 2
Title: The Pragmatic Programmer
Author: Andrew Hunt
Category: Software Engineering
Status: Available
ISBN: 9780135957059
Publisher: Addison Wesley
Pages: 352
Are you sure you want to borrow this book? (y/n): y

>>> BORROW DETAILS <<<
Issue Date: 2026-06-16
Due Date: 2026-06-30
Error: Resource Error:
Borrow limit reached! Your Silver membership allows 2 books. Return some books to borrow more.
```


Returning a Borrowed Resource

```
Enter your choice: 6
Resource ID to return: 3
Confirm return now? (y/n): y

RETURN SUMMARY
-----
Book: Design Patterns
Borrowed Date: 2026-06-16
Due Date: 2026-06-30
Returned Date: 2026-06-16
Returned on time! No late fees.
Return successful!
Return recorded. Admin will verify the resource condition and apply any damage fees if needed.
```

```
[6] [*] RETURN CONFIRMED - 2026-06-16
Return confirmed for Resource ID 3 (Design Patterns).
-----
```

Overdue Fine Calculation



```
Enter your choice: 4
Physical Resource ID to borrow: 5

>>> BORROW CONFIRMATION <<<

=====
PRINTED BOOK DETAILS
=====
ID: 5
Title: Code Complete
Author: Steve McConnell
Category: Software Engineering
Status: Available
ISBN: 9780735619678
Publisher: Microsoft Press
Pages: 960
Are you sure you want to borrow this book? (y/n): y

>>> BORROW DETAILS <<<
Issue Date: 2026-05-16
Due Date: 2026-05-30
Borrow successful!
Book: Code Complete
Due Date: 2026-05-30
Late Fee: Rs. 10 per day after due date

Borrow processed successfully!
A notification has been sent to your notifications tab.
```

```
[7] [*] BORROW APPROVED - 2026-05-16
You have successfully borrowed 'Code Complete'. Due date: 2026-05-30. Please return on time to avoid late fees.
-----
```



Account Balance: Rs. 340 (Credit)

```
===== BORROW RECORD =====
Title      : Code Complete
Author     : Steve McConnell
Category   : Software Engineering
Record ID  : 4
Resource ID : 5
Issue Date : 2026-05-16
Due Date   : 2026-05-30
Return Date : Not returned
Status     : Overdue
Fine       : 150
```

```

Enter your choice: 6
Resource ID to return: 5
Confirm return now? (y/n): y

RETURN SUMMARY
-----
Book: Code Complete
Borrowed Date: 2026-05-16
Due Date: 2026-05-30
Returned Date: 2026-06-14
LATE FEE: Rs. 150 (15 days overdue)
Fine settled using account credit. Remaining balance: Rs. 190
Return successful!
Return recorded. Admin will verify the resource condition and apply any damage fees if needed.

```

```

[8] [*] FINE APPLIED - 2026-06-14
A late fine of Rs. 150.000000 was applied for Resource ID 5.
-----

[9] [*] RETURN CONFIRMED - 2026-06-14
Return confirmed for Resource ID 5 (Code Complete).
-----

```

Ratings and Review

```

=====
RATE & REVIEW A BOOK
=====
Resource ID: 5

Book: Code Complete by Steve McConnell
Category: Software Engineering
Rating (1-5): 4
Would you like to add a review? (y/n): y
Review (max 200 chars): Excellent Book!!

Review submitted successfully! Thank you for sharing your feedback.

```

Test Case 5 Report Generation System

Fine management report

9. Generate Issued/Overdue Resource Report

```

Choice: 9

===== ISSUED AND OVERDUE RESOURCES =====
Record ID : 1
Resource ID : 44 (Fundamentals of Database Systems)
Borrowed By : Manahil Imran (ID 3)
Issue Date : 2026-05-14
Due Date : 2026-05-28
Status : Overdue
Overdue Days: 16 | Fine Estimate: Rs. 0 (eBook)
Current Fine: Rs. 0
-----

```

Borrow Management Report

8. Generate All Member Borrow Reports

```

===== MEMBER BORROWING SUMMARY =====
Member: Manahil Imran (ID 3)
Current Borrows: 1
--- Current Borrowed Resources ---
Resource ID: 44 | Issue: 2026-05-14 | Due: 2026-05-28 | OVERDUE

--- Full Borrow History ---

===== BORROW RECORD =====
Title : Fundamentals of Database Systems
Author : Ramez Elmasri
Category : Databases
Record ID : 1
Resource ID : 44
Issue Date : 2026-05-14
Due Date : 2026-05-28
Return Date : Not returned
Status : Overdue
Fine : 0

```

```
===== BORROW RECORD =====
Title       : Database System Concepts
Author      : Abraham Silberschatz
Category    : Databases
Record ID   : 2
Resource ID : 9
Issue Date  : 2026-05-17
Due Date    : 2026-05-31
Return Date : 2026-06-16
Status      : Returned
Fine        : 160
```

```
===== BORROW RECORD =====
Title       : Design Patterns
Author      : Erich Gamma
Category    : Software Engineering
Record ID   : 3
Resource ID : 3
Issue Date  : 2026-06-16
Due Date    : 2026-06-30
Return Date : 2026-06-16
Status      : Returned
Fine        : 0
```

```
===== BORROW RECORD =====
Title       : Code Complete
Author      : Steve McConnell
Category    : Software Engineering
Record ID   : 4
Resource ID : 5
Issue Date  : 2026-05-16
```

```
Issue Date : 2026-05-16
Due Date    : 2026-05-30
Return Date : 2026-06-14
Status      : Returned
Fine        : 150
```

```
===== MEMBER BORROWING SUMMARY =====
Member: Mahnoor Naeem (ID 5)
Current Borrows: 1
--- Current Borrowed Resources ---
Resource ID: 5 | Issue: 2026-06-14 | Due: 2026-06-28

--- Full Borrow History ---
```

```
===== BORROW RECORD =====
Title       : Code Complete
Author      : Steve McConnell
Category    : Software Engineering
Record ID   : 5
Resource ID : 5
Issue Date  : 2026-06-14
Due Date    : 2026-06-28
Return Date : Not returned
Status      : Borrowed
Fine        : 0
```

7. INDIVIDUAL CONTRIBUTIONS OF EACH MEMBER IN THE PROJECT

This project was developed as a collaborative effort by three team members. The workload was distributed fairly among all members, with each person being responsible for a major module of the Library Management System. Although each member focused on a specific part of the project, continuous coordination and teamwork were maintained throughout the development process to ensure smooth integration of all modules into a single fully functional system.

UML Diagram was made together.

MEMBER 01 **Rumaisa Waseem CS-25052**

User & Account Management Module

Responsibilities and contributions include:

- Designed and implemented the complete User Management System
- Created the abstract base class User and derived classes Member and Admin
- Implemented secure login, logout, and authentication mechanisms
- Worked on password hashing and salting for secure credential management
- Implemented role-based access handling for Members and Admins
- Added validation checks for all user inputs
- Implemented exception handling for invalid operations and login attempts
- Developed the UserManager class for managing user-related operations
- Implemented file handling for storing and retrieving user records
- Assisted in debugging, integration of main, and testing of the complete system

MEMBER 02: **Laiba Qasim CS-25069**

Resource Management Module

Responsibilities and contributions include:

- Designed and implemented the complete Resource Management System
- Created the abstract base class Resource and derived classes Book and Ebook
- Implemented library inventory and resource availability management
- Added resource search and filtering functionalities
- Managed file handling for storing and retrieving resource data
- Developed the LibarayManager class for managing resources-related operations
- Implemented resource categorization and availability tracking
- Added validation checks for user inputs
- Assisted in testing, debugging, and integration of modules with the Library system

MEMBER 03: Aleena Khalid CS-25056

Borrowing, Fine Management & System Integration Module

Responsibilities and contributions include:

- Designed and implemented the BorrowRecord management system
- Developed the FineManager module for overdue fine calculation
- Created the ReportGenerator module for generating reports and summaries
- Implemented the LibraryManager , manages resources relaed operations.
- Managed borrowing and returning operations
- Implemented due-date tracking and borrowing validation
- Added validation checks for all user inputs
- Handled composition and aggregation relationships between classes
- Assisted in managing notifications and transaction tracking